

Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Радіотехнічний факультет
Кафедра радіотехнічних систем

ЗВІТ

до лабораторної роботи №8

з дисципліни «Програмовані засоби в інтелектуальній радіоелектронній техніці»

«РЕАЛІЗАЦІЯ КОНСОЛЬНОЇ ВЗАЄМОДІЇ З ВБУДОВАНОЮ СИСТЕМОЮ»

Студент: Кравченко Артем

Група: РЕ-41

Викладач: _____

Київ 2026

Мета роботи

Ознайомитись з особливостями розроблення вбудовуваного інтерфейсу користувача на основі послідовного порту (UART) для реалізації консольного меню. Набути навичок використання переривань для обробки асинхронного введення даних через консоль. Навчитися взаємодіяти з зовнішніми модулями (наприклад, GSM/GPS) за допомогою AT-команд та реалізувати автоматичне розпізнавання (парсинг) відповідей.

Короткий план виконання роботи

1. Виконано ініціалізацію модуля USART для організації консольного обміну даними через UART.
2. Реалізовано діалогову структуру меню.
3. Модифіковано прийом даних так, щоб команди приймались через переривання RCIE, буферизувались та розпізнавались лише після надходження символу завершення рядка.
4. Реалізовано передачу аргументу разом з командою.
5. Створено інтерфейс за допомогою якого можливо керувати PWM модулем МК дистанційно за допомогою консолі (з'єднання USART).

Код програми

```
1 #include <pl8f4550.h>
2 #include <delays.h>
3
4 #define K1 0x01
5 #define K2 0x02
6 #define K3 0x04
7 #define PIN_Keys PORTE
8 #define PIN_K1 PORTB
9 #define PIN_K2 PORTB
10 #define PIN_K3 PORTB
11 #define Keypad (K1 | K2 | K3)
12
13
14 unsigned int counter;
15 unsigned int keycntr;
16 struct flagsStruct
17 {
18     unsigned int clicking;
19     unsigned int pushing;
20 };
21
22 struct flagsStruct flags;
23 unsigned char press_code;
24 unsigned int adc_value;
25 char buffer[6];
26 char command[100];
27 char commandCharCounter;
28 const char cmd_pwmmon[] = "pwmmon";
29 const char cmd_pwmoff[] = "pwmoff";
30 const char cmd_pwmset[] = "pwmset";
31 const char cmd_pwmmon_response[] = "PWM is on\r";
32 const char cmd_pwmoff_response[] = "PWM is off\r";
33 const char cmd_pwmset_response[] = "PWM is set\r";
34 unsigned char k;
35
36 void systemInit(void);
37 //void TMR0Init(void);
38 void USARTInit(void);
39 //void ADCInit(void);
40 void PWMInit(void);
41
42 //void ADCInit(void);
43 void PWMInit(void);
44
45 //void TMR0Interrupt(void);
46
47 void sendCharUSART(char);
48 void sendWordUSART(char*);
49 void numberToWord(unsigned int, char*);
50
51 char readCharUSART(void);
52 void operateCommand(char*);
53 char compareWithWord(char*, const char*);
54
55 //unsigned char keyboard(void);
56 void PWMOn(void);
57 void PWMOff(void);
58 void PWMSetup(unsigned char);
59
60 extern void _startup (void);
61 void ISRHigh(void);
62
63 #pragma code main=0x102A
64 void main(void)
65 {
66     TRISB = 0;
67     ADCON1 = 0x0F;
68     counter = 1;
69     keycntr = 0;
70     adc_value = 0;
71
72     systemInit();
73     // TMR0Init();
74     USARTInit();
75     // ADCInit();
76     PWMInit();
77
78     while (1)
79     {
80         sendCharUSART('\f');
81         if (press_code & 128) sendCharUSART('L');
82         if (press_code)
83
84
85         {
86             unsigned char multiplier;
87
88             sendCharUSART('B');
89             sendCharUSART((press_code & 7) | '0');
90             multiplier = 1 << ((press_code & 7) - 1);
91             multiplier *= (press_code & 128) ? (8) : 1;
92             sendCharUSART('M');
93             numberToWord(multiplier, buffer);
94             sendWordUSART(buffer);
95             PWMSetup(multiplier);
96         }
97
98         for (k = 0; k < 1; k++)
99         {
100             Delay10KTCYx(250);
101         }
102     }
103
104 #pragma code interrupt_high_vector=0x1008
105 void interrupt_high_vector(void)
106 {
107     _asm goto ISRHigh _endasm
108 }
109
110 #pragma code
111 #pragma interrupt ISRHigh
112 void ISRHigh(void)
113 {
114     if (INTCONbits.TMR0IF)
115     {
116         INTCONbits.TMR0IF = 0;
117         TMR0Interrupt();
118     }
119
120     if (PIR1bits.RCIF)
121     {
122         char receivedChar = readCharUSART();
123
124         PIR1bits.RCIF = 0;
125         if (receivedChar == '\r')
126         {
127             command[commandCharCounter] = '\0';
128             operateCommand(command);
129             commandCharCounter = 0;
130         }
131         else
132         {
133             command[commandCharCounter++] = receivedChar;
134         }
135     }
136
137     Delay10KTCYx(250);
138 }
139
140 void systemInit(void)
141 {
142     INTCONbits.GIE = 1;
143     INTCONbits.PEIE = 1;
144 }
145
146 //void TMR0Init(void)
147 //{
148     // TOCON = 0b10010001;
149     // INTCONbits.TMR0IE = 1;
150     // INTCON2bits.TMR0IP = 1;
151 //}
152
153 void USARTInit(void)
154 {
155     SPBRGH = 0;
156     BAUDCONbits.BRG16 = 0;
157
158     SPBRG = 77;
159     TXSTAbits.BRGH = 0;
160     TXSTAbits.SYNC = 0;
161 }
```

```

154     SPBRG = 77;
155     TXSTABits.BRGH = 0;
156     TXSTABits.SYNC = 0;
157     RCSTABits.SPEN = 1;
158     RCSTABits.CREN = 1;
159
160     TXSTABits.TX9 = 0;
161     TXSTABits.TXEN = 1;
162
163     PIR1bits.RCIE = 1;
164     IPR1bits.RCIP1 = 1;
165 }
166
167 //void ADCInit(void)
168 //{
169 //    ADCON0 = 0b00000001;
170 //    ADCON1 = 0b00001110;
171 //    ADCON2 = 0b00101110;
172 //    ADCON0bits.GO = 1;
173 //}
174
175 void PWMInit(void)
176 {
177     CCP1CON = 0b00001100;
178     T2CON = 0b00000000;
179     TRISCbits.RC1 = 0;
180     TRISCbits.RC2 = 0;
181     TRISBbits.RB3 = 0;
182 }
183
184 void PWMOn(void)
185 {
186     T2CONbits.TMR2ON = 1;
187     // numberToWord(PR2, buffer);
188     // sendWordUSART(buffer);
189     if (PR2 <= 1 || PR2 >= 240)
190     {
191         // sendCharUSART('o');
192     }

```

```

228 // sendCharUSART('!');
229 // numberToWord(PR2, buffer);
230 // sendWordUSART(buffer);
231
232 else if (multiplier >= 64 && multiplier < 256)
233 {
234     T2CONbits.T2CKPS = 0;
235     PR2 = ((char)(240 * 16 / multiplier)) - 1;
236 }
237 CCP1RL = (char)((PR2 + 1) / 2);
238 // numberToWord(CCP1RL, buffer);
239 // sendWordUSART(buffer);
240 T2CONbits.TMR2ON = pwm_state;
241
242 else
243 {
244     T2CONbits.TMR2ON = 0;
245 }
246
247 void sendCharUSART(char data)
248 {
249     while(TXSTABits.TRMT == 0);
250     TXREG = data;
251 }
252
253 void sendWordUSART(char* word)
254 {
255     int i = 0;
256     // sendCharUSART('\n');
257     while(word[i] != '\0' && i != 100)
258     {
259         sendCharUSART(word[i]);
260         i++;
261     }
262     // sendCharUSART('\n');
263 }
264
265 void numberToWord(unsigned int number, char* word)
266 {

```

```

191 {
192     // sendCharUSART('o');
193     T2CONbits.T2CKPS = 2;
194     PR2 = 59;
195     CCP1L = (PR2 + 1) / 2;
196 }
197
198 void PWMOff(void)
199 {
200     T2CONbits.TMR2ON = 0;
201 }
202
203 //void TMR0Interrupt(void)
204 //{
205 //    unsigned char current_press_code = keyboard();
206 //    if (current_press_code) press_code = current_press;
207 //}
208
209 void PWMSetup(unsigned char multiplier)
210 {
211     char pwm_state = T2CONbits.TMR2ON;
212     if (multiplier)
213     {
214         T2CONbits.TMR2ON = 0;
215         if (multiplier < 16)
216         {
217             T2CONbits.T2CKPS = 2;
218             PR2 = ((char)(240 / multiplier)) - 1;
219             // numberToWord(PR2, buffer);
220             // sendWordUSART(buffer);
221         }
222         else if (multiplier >= 16 && multiplier < 64)
223         {
224             T2CONbits.T2CKPS = 1;
225             PR2 = ((char)(240 * 4 / multiplier)) - 1;
226             // sendCharUSART('!');
227             // numberToWord(PR2, buffer);
228             // sendWordUSART(buffer);
229         }
230     }

```

```

265 void numberToWord(unsigned int number, char* word)
266 {
267     register unsigned char n;
268     for(n = 0; n <= 4; n++)
269     {
270         word[n] = '0';
271     }
272     for(n = 0; n <= 4; n++)
273     {
274         word[4-n] = (number % 10) + '0';
275         number /= 10;
276         if (!number) break;
277     }
278 }
279
280 int wordToNumber(char* word)
281 {
282     char i = 0;
283     int number = 0;
284     while (word[i] != '\0')
285     {
286         if (word[i] < '0' || word[i] > '9')
287         {
288             number = 0;
289             break;
290         }
291         number *= 10;
292         number += word[i] - '0';
293         i++;
294     }
295     // sendCharUSART('?');
296     // sendCharUSART(number);
297     return number;
298 }
299
300 char readCharUSART(void)
301 {
302     char receivedByte;
303     receivedByte = RCREG;
304     sendCharUSART(receivedByte);
305 }

```

```

302 {
303     char receivedByte;
304     receivedByte = RCREG;
305     sendCharUSART(receivedByte);
306     return receivedByte;
307 }
308
309 void operateCommand(char* command)
310 {
311     char i = 0;
312     char j = 0;
313     char k = 0;
314
315     char command_formatted[50];
316     char command_body[20];
317     char command_argument[20];
318
319     unsigned long frequency;
320     unsigned char multiplier;
321
322     // while (command[i])
323     // {
324     //     sendCharUSART(command[i]);
325     //     i++;
326     // }
327
328     while (command[i] != '\0')
329     {
330         if (command[i] == '\b' && j > 0)
331         {
332             j--;
333             i++;
334             continue;
335         }
336         command_formatted[j] = command[i];
337         i++;
338         j++;
339     }
340     command_formatted[j] = '\0';
341
342     j++;
343     command_formatted[j] = '\0';
344     i = 0;
345     j = 0;
346     while (command_formatted[i] != ' ' && command_formatted[i] != '\0')
347     {
348         command_body[j] = command_formatted[i];
349         i++;
350         j++;
351     }
352     command_body[j] = '\0';
353     if (command_formatted[i] != '\0') i++;
354     while (command_formatted[i] != '\0')
355     {
356         command_argument[k] = command_formatted[i];
357         i++;
358         k++;
359     }
360     command_argument[k] = '\0';
361     // sendWordUSART(command_body);
362     // sendCharUSART('\n');
363     // sendWordUSART(command_argument);
364     // sendCharUSART('\n');
365
366     if (compareWithWord(command_body, cmd_pwmmon))
367     {
368         PWMOn();
369         sendWordUSART(cmd_pwmmon_response);
370     }
371     else if (compareWithWord(command_body, cmd_pwmoff))
372     {
373         PWMOff();
374         sendWordUSART(cmd_pwmoff_response);
375     }
376     else if (compareWithWord(command_body, cmd_pwmset))
377     {
378         frequency = wordToNumber(command_argument);
379         if (frequency < 4) frequency = 4;
380         if (frequency >= 800) frequency = 799;
381
382         multiplier = frequency / 3.125;
383         if (multiplier < 1) multiplier = 1;
384         if (multiplier > 255) multiplier = 255;
385
386         // sendCharUSART('^');
387         // sendCharUSART(multiplier);
388         // numberToWord(multiplier, buffer);
389         // sendWordUSART(buffer);
390         // sendCharUSART('\n');
391         PWMSetup(multiplier);
392         sendWordUSART(cmd_pwmset_response);
393     }
394 }
395
396 char compareWithWord(char* word1, const char* word2)
397 {
398     int i = 0;
399     while (1)
400     {
401         if (word1[i] == '\0' || word2[i] == '\0') return (word1[i] == '\0') && (word2[i] == '\0');
402         if (word1[i] != word2[i]) return 0;
403         i++;
404     }
405 }

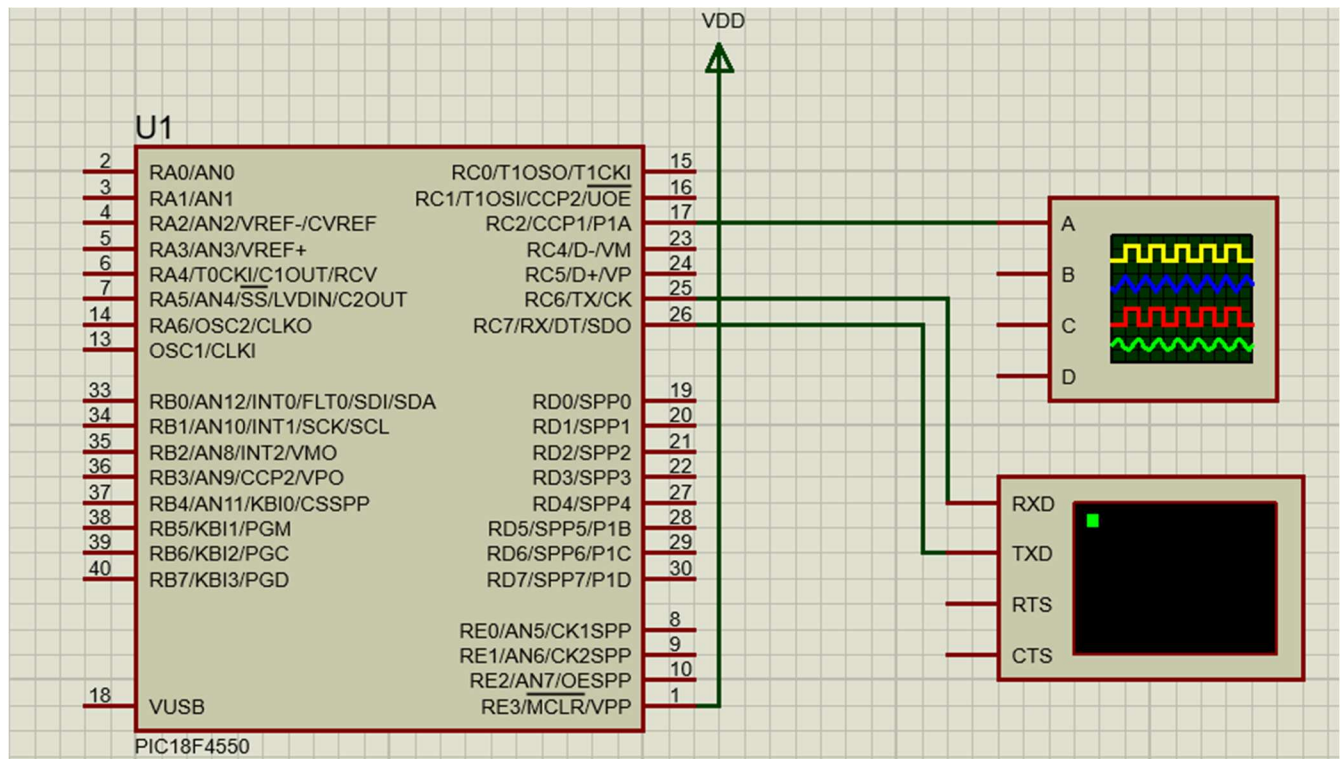
```

```

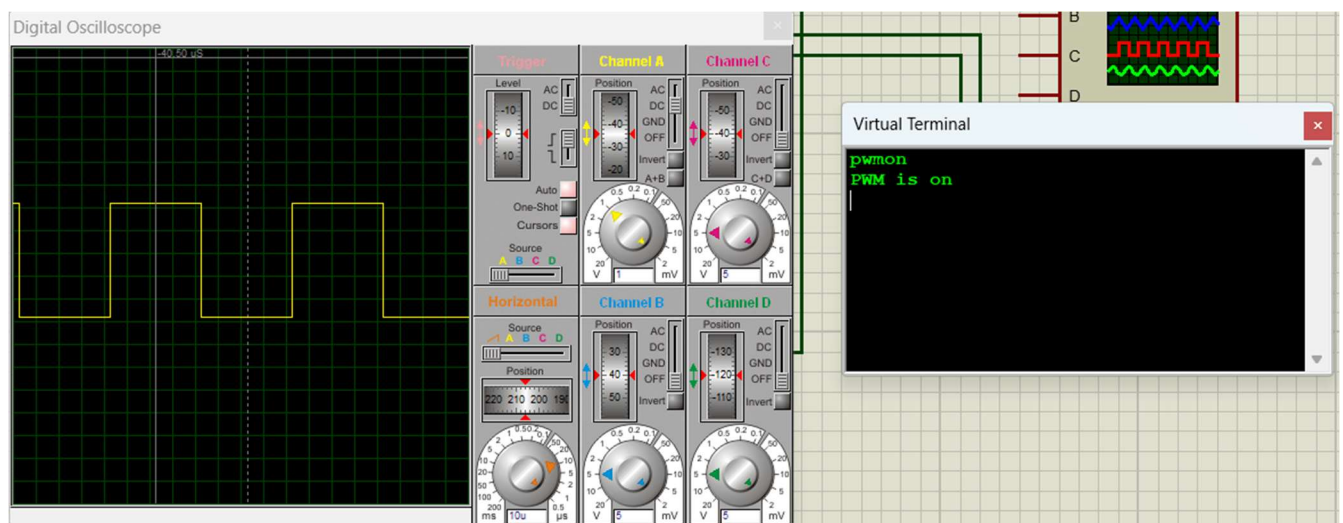
379
380     multiplier = frequency / 3.125;
381     if (multiplier < 1) multiplier = 1;
382     if (multiplier > 255) multiplier = 255;
383
384     // sendCharUSART('^');
385     // sendCharUSART(multiplier);
386     // numberToWord(multiplier, buffer);
387     // sendWordUSART(buffer);
388     // sendCharUSART('\n');
389     PWMSetup(multiplier);
390     sendWordUSART(cmd_pwmset_response);
391 }
392
393 char compareWithWord(char* word1, const char* word2)
394 {
395     int i = 0;
396     while (1)
397     {
398         if (word1[i] == '\0' || word2[i] == '\0') return (word1[i] == '\0') && (word2[i] == '\0');
399         if (word1[i] != word2[i]) return 0;
400         i++;
401     }
402 }

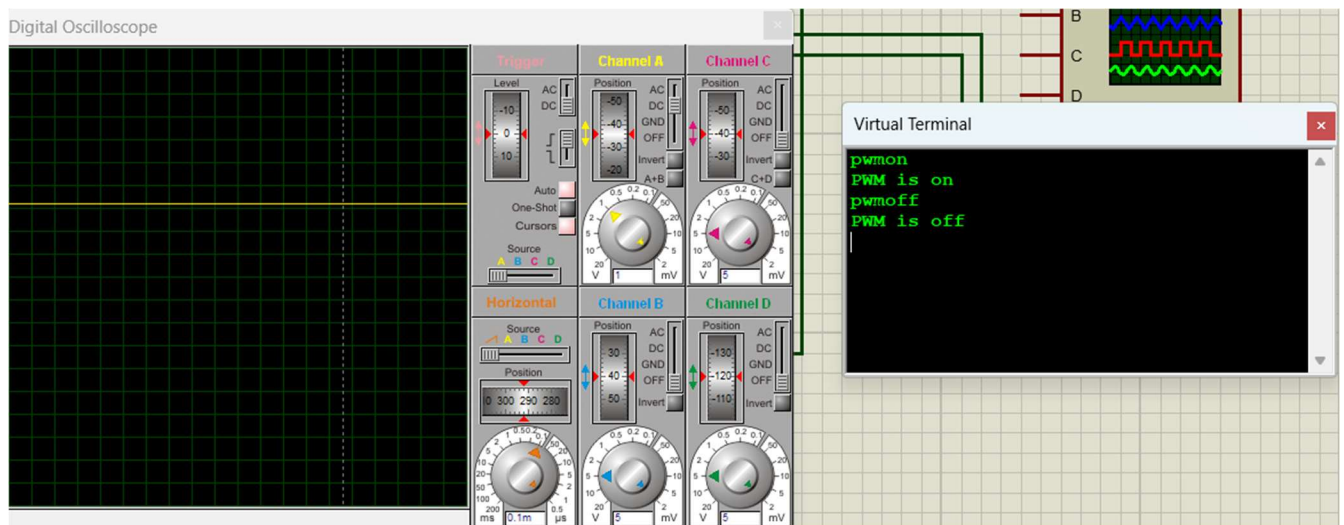
```

Результат виконання у середовищі Proteus (детальніше у демонстрації):

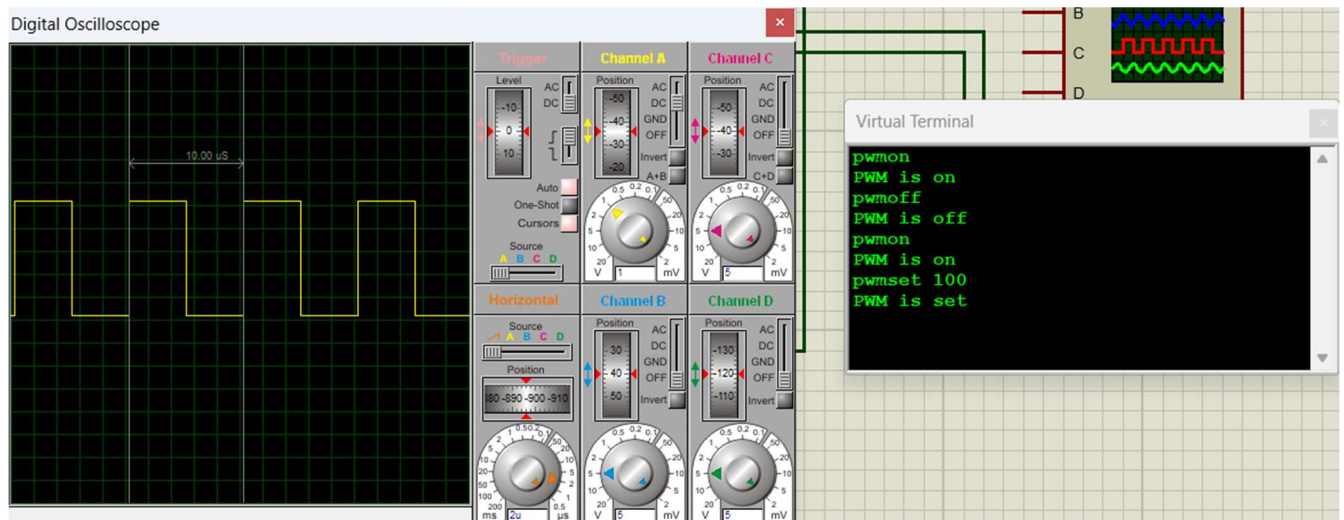


Користувач може вмикати, вимикати та налаштовувати частоту згенерованого ШІМ сигналу.





Приймання даних через UART реалізовано з буферизацією, тому команди обробляються лише після надходження символу завершення рядка. Це дозволяє коректно розпізнавати команди типу `pwwon` та `pwwoff`. Також додано можливість передачі аргументів команди (через символ пробілу):



`pwmset «frequency»` налаштовує період ШІМ відповідно до заданої частоти, де `frequency` – частота в кГц (роздільна здатність – 3.125 кГц).

У прикладі вище частота = 100 кГц, відповідно період імпульсу = $1 / 100 \text{ кГц} = 10 \text{ мкс}$, що підтверджується на скріншоті.

Контрольні запитання

1. Як відрізняється підхід щодо прийому даних через консоль UART: а) через опитування RCIF; б) через переривання RCIE?

При опитуванні RCIF програма постійно перевіряє наявність нового символу і витрачає на це процесорний час. При RCIE символ приймається через переривання, зчитується з RCREG і зберігається у буфер, тому основна програма може паралельно виконувати інші задачі.

2. Навіщо у проектах з програмним завантажувачем вектори переривань переносяться на нові адреси? Поясніть необхідність запису 0x10 у PCLATH для переходу на 0x1000.

Bootloader займає початок пам'яті програм, тому основна програма і вектори переривань переносяться на нові адреси. Значення 0x10 у PCLATH потрібне для формування правильної старшої частини адреси переходу на 0x1000.

3. Які переваги має використання циклічного буфера (FIFO) перед простим лінійним масивом для прийому послідовних даних?

FIFO дає змогу безперервно приймати символи у порядку їх надходження без зсуву даних у пам'яті. Це зручніше для UART, бо зменшує навантаження на програму і знижує ризик втрати символів.

4. Як повинна бути організована функція парсингу відповідей AT-команд, щоб надійно ідентифікувати кінцеві маркери "OK" або "ERROR"?

Функція повинна аналізувати буфер прийому після накопичення повної відповіді та перевіряти наявність підрядків OK\r\n або ERROR\r\n. Якщо знайдено OK, команда виконана успішно, якщо ERROR — формується повідомлення про помилку або повтор запиту.

5. У чому полягає роль бітів IPEN, GIEH, GIEL та INTxIP у налаштуванні переривань?

IPEN вмикає режим пріоритетних переривань. GIEH дозволяє високопріоритетні

переривання, GIEL — низькопріоритетні, а біти INTxIP задають пріоритет зовнішніх переривань.

6. Які AT-команди є стандартизованими?

До стандартизованих належать базові AT-команди, наприклад AT для перевірки зв'язку, ATE0 або ATE1 для керування ехом, ATІ для інформації про пристрій, AT+CSQ для оцінки рівня сигналу. Розширені команди можуть залежати від виробника модуля.